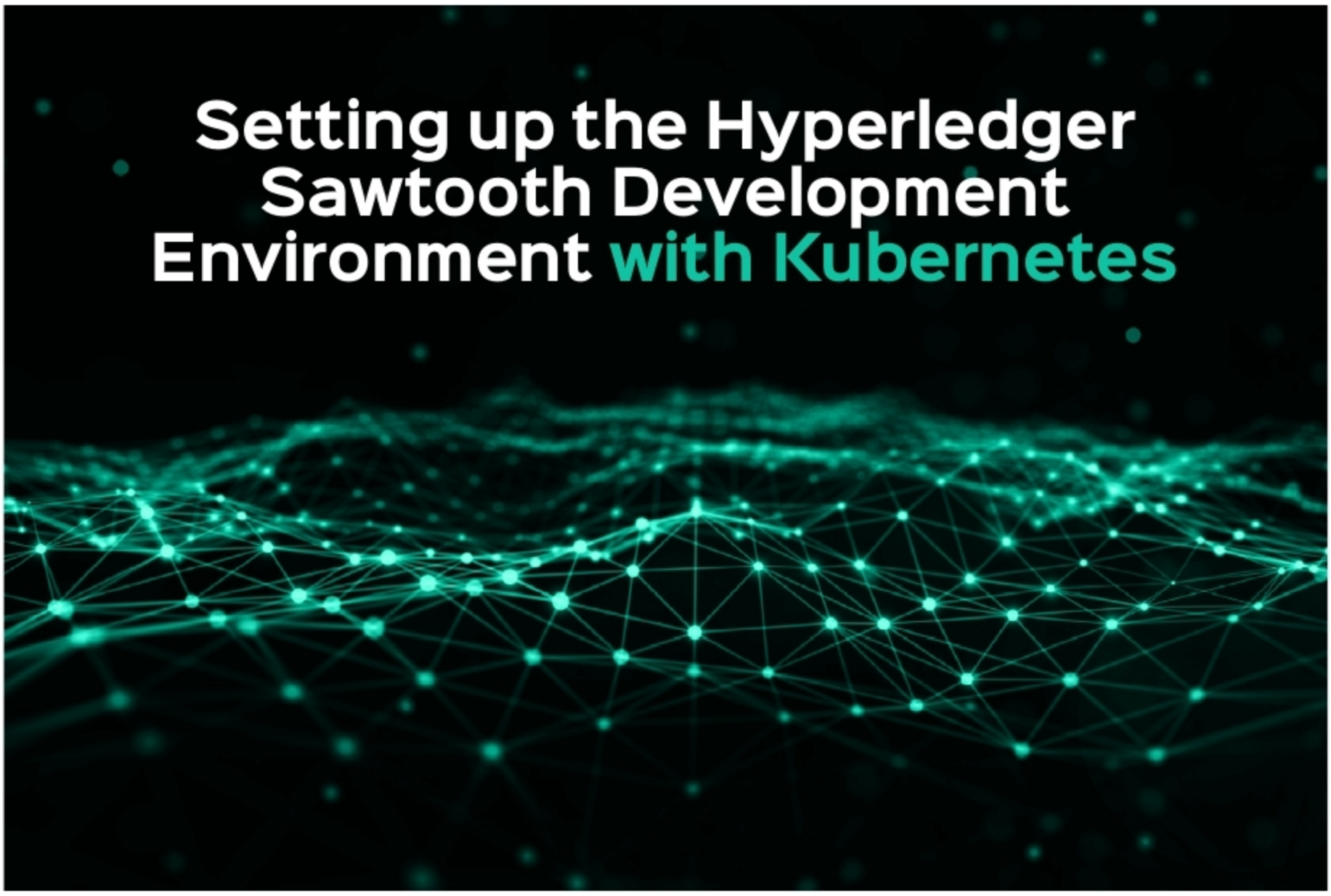
Setting up the Hyperledger Sawtooth Development Environment with Kubernetes



This article is aimed at open source enthusiasts with an interest in blockchain technologies and who have working knowledge of the Hyperledger Sawtooth project.

In this article, we will create a single Hyperledger Sawtooth validator node with Kubernetes.

The environment required for this uses Minikube to deploy Sawtooth as a containerised application in a local Kubernetes cluster inside a virtual machine (VM) on your computer. After completing this procedure, you will have the environment required for evaluating a Hyperledger Sawtooth network or use the Sawtooth SDKs. This article assumes you are familiar with Kubernetes and are setting up the single-node Minikube.

Prerequisites

This application development environment requires *kubectl* and Minikube with a supported VM hypervisor, such as VirtualBox, HyperV, etc.

The application development environment

This application development environment is a single validator node that is running a validator, a REST API, and three transaction processors. This environment uses dev mode consensus and parallel transaction processing.

The Kubernetes cluster has one pod with a container for each Sawtooth component. After the container begins to run, you can use the Kubernetes dashboard to view the pod status, the container names, Sawtooth log files, and more.

This example environment includes the following transaction processors:

- *Settings* handles Sawtooth's on-chain settings. The *sawtooth-settings-tp* transaction processor is required for this environment.
- *IntegerKey* is a basic application (also called a transaction family) that introduces Sawtooth functionality. The *sawtooth-intkey-tp-python* transaction processor works with the *int-key* client, which has shell commands to perform integer-based transactions.
- XO is a simple application for playing a game of tic-tac-toe on the blockchain. The *sawtooth-xo-tp-python* transaction processor works with the *xo* client, which has shell commands to define players and play a game. XO will be described in a later tutorial.